RoomRadar QC — Manual

What The App Does

RoomRadar QC is a web app for Queens College students to find available classrooms in real time. Students search by day, time range, and building. Admins upload a CSV file of the semester schedule to populate the database.

Team

Member	Role
Darshan	Backend Developer / Database Engineer
Kyro	Frontend Developer
Joseph	Tester / QA

Tech Stack

- **Frontend:** React + Vite + Material UI
- **Backend:** Python + Flask
- **Database:** MySQL hosted on AWS EC2
- **Auth:** Firebase (frontend login) + Firebase Admin SDK (backend token verification)

Project Structure

```
CSCI-370-Final-Project/
├── frontend/                  React frontend
│   └── src/
│       ├── pages/
│       │   ├── Home.jsx
│       │   ├── FindRooms.jsx       search rooms by day/time/building
│       │   ├── RoomDetails.jsx     view room schedule
│       │   ├── AdminLogin.jsx      Firebase login
```

```
│       │   └── AdminDashboard.jsx  CSV upload
│       ├── components/
│       │   ├── Navbar.jsx
│       │   └── ProtectedAdminRoute.jsx
│       └── firebase.js
├── backend/
│   ├── app.py                  Flask entry point, Firebase Admin init
│   ├── config.py               EC2 MySQL connection settings
│   ├── models.py               Creates DB tables on startup
│   ├── requirements.txt        Python dependencies
│   └── routes/
│       ├── admin.py            POST /api/admin/semester (CSV upload)
│       └── rooms.py            GET /api/rooms/search, GET
/api/rooms/<id>/schedule
├── Rooms.csv                   Queens College schedule data
└── CONTEXT.md                  This file
```

API Endpoints

Method	Endpoint	Description
POST	`/api/admin/semester`	Admin uploads CSV → populates DB
GET	`/api/rooms/search`	Find available rooms by day/time/building
GET	`/api/rooms/<id>/schedule`	Get full schedule for a room

Search Parameters

- `day` — Monday, Tuesday, Wednesday, Thursday, Friday
- `starttime` — HH:MM format
- `endtime` — HH:MM format
- `building` — building code (PH, KY, SB, RE) or "All"

Database Schema

```
CREATE TABLE semesters (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50)
```

```
);

CREATE TABLE rooms (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_code VARCHAR(20),
    building VARCHAR(100),
    capacity INT
);

CREATE TABLE schedules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    room_id INT,
    semester_id INT,
    course_name VARCHAR(100),
    instructor VARCHAR(100),
    day VARCHAR(20),
    start_time TIME,
    end_time TIME,
    FOREIGN KEY (room_id) REFERENCES rooms(id),
    FOREIGN KEY (semester_id) REFERENCES semesters(id)
);
```

AWS EC2 Setup

- **EC2 IP:** 18.223.123.14
- **MySQL Port:** 3306 (open to 0.0.0.0/0 in security group)
- **Database:** school_project
- **MySQL User:** darshan
- bind-address set to 0.0.0.0 in /etc/mysql/mysql.conf.d/mysqld.cnf

---

CSV Format (Rooms.csv)

Columns used from the Queens College schedule CSV:

- `Description` — course name
- `Day` — class day (M, T, W, TH, F or combinations like "T, TH")
- `Time` — time range (e.g. "9:10 AM - 12:00 PM")
- `Instructor` — instructor name
- `Location` — room code (e.g. "PH 110")

- `Limit` — room capacity
- `Mode of Instruction` — only "In-Person" rows are imported

Rows skipped:

- Online classes (Location starts with "OL")
- Missing/empty time values
- Non in-person classes

Authentication Flow

1. Admin logs in via Firebase on the frontend (AdminLogin.jsx)
2. Firebase returns an ID token stored in localStorage as "adminToken"
3. AdminDashboard.jsx sends the token as `Authorization: Bearer <token>`
4. Flask backend verifies the token using Firebase Admin SDK
5. Returns 401 if token is missing or invalid

Firebase Service Account Key

- File: `roomradar-qc-firebase-adminsdk-fbsvc-2ae032eef0.json`
- Location: `backend/` folder
- NOT committed to GitHub (in .gitignore)
- Must be shared manually between team members

Running The Project

Backend

```
cd backend
pip install -r requirements.txt
python app.py
```

Runs on http://localhost:5000

Frontend

```
cd frontend
npm install
npm run dev
```

Runs on http://localhost:5173

Dependencies

Backend (requirements.txt)

- flask
- flask-cors
- mysql-connector-python
- firebase-admin
- gunicorn (For Render Hosting Service)

Frontend (package.json)

- react
- react-router-dom
- @mui/material
- firebase

Known Limitations

- The frontend is currently configured for local development and sends API requests to `http://127.0.0.1:5000` or `http://localhost:5000`. When the backend is deployed to a remote server, these API URLs must be updated to use the deployed backend address.
- Firebase admin credentials are not included in the GitHub repository for security reasons. Each developer who needs to run admin/backend features locally must be given the required service account key separately.
- The room availability logic depends on the day format used in the uploaded CSV file. Values such as `M`, `T`, `W`, `TH`, `F`, `S`, and `SU` must match the expected format for the system to process schedule data correctly.
- Room availability is based only on course schedule data. The system cannot confirm whether a room is physically occupied, locked, reserved for an unofficial event, or otherwise unavailable.
- The accuracy of the search results depends on the accuracy and completeness of the uploaded semester schedule data.

- The system is designed specifically for Queens College schedule data and building codes. Additional changes may be needed before using the system for another campus or institution.

Key Bugs Fixed

1. **Duplicate room records**
   The CSV upload process was creating a new room record for every class row, even when the room already existed. This caused duplicate rooms to appear in the database and search results. The issue was fixed by checking whether a room already exists before inserting a new room record.

2. **Time serialization error**
   MySQL `TIME` columns were returned as Python `timedelta` objects, which could not be directly converted to JSON. This caused API responses to fail when returning schedule times. The issue was fixed by converting time values into strings before sending them in the JSON response.

3. **Incorrect frontend API requests**
   Some frontend requests were being sent to relative URLs, which caused the browser to send the request to the Vite development server instead of the Flask backend. This was fixed by using the full backend API URL, such as `http://127.0.0.1:5000` or `http://localhost:5000`.

4. **CSV upload file handling issue**
   The CSV upload process had issues with `SpooledTemporaryFile` and `io.TextIOWrapper` in Python 3.8. This caused uploaded files to be read incorrectly in some cases. The issue was fixed by reading the uploaded file as bytes, decoding it, and wrapping it with `io.StringIO`.

5. **Database connection issue from remote services**
   MySQL was only listening on `localhost`, which prevented outside services from connecting to the database. This was fixed by updating the MySQL configuration and setting `bind-address` to `0.0.0.0` in `mysqld.cnf`.

6. **Frontend-to-backend local address mismatch**
   The frontend initially used `localhost:5000`, but the Flask backend was running on `127.0.0.1:5000` in the local environment. This caused requests to fail in some cases. The issue was fixed by updating frontend fetch requests to use the backend address shown in the Flask terminal.

7. **Room details page failing to load schedules**
   The RoomDetails page was not properly reaching the backend schedule endpoint, causing the page to display a failed schedule loading message. This was fixed by updating the API URL and ensuring the route matched the backend endpoint for room schedule details.

8. **Unnecessary seconds in displayed schedule times**
   Schedule times were displayed with seconds, such as `11:53:00`, which made the UI look less clean. This was fixed by formatting schedule times before displaying them on the RoomDetails page.

9. **Generated frontend files appearing in Git**
   Vite cache files such as `.vite`, along with generated folders like `node_modules` and `dist`, appeared in the project workspace and could accidentally be committed. This was fixed by updating `.gitignore` to exclude generated frontend and backend files.

10. **Weekend search options missing**
    The original search form only focused on weekday options. Since the schedule data contained Saturday and Sunday classes, weekend search options were added so users can search room availability for all days represented in the schedule data.

11. **Admin page accessible through direct URL**
    The admin dashboard could be accessed by manually typing the admin route into the browser URL, even if the user was not properly authenticated. This created an access control issue for admin-only pages. The issue was fixed by adding protected route logic so unauthenticated users are redirected to the admin login page before they can access admin pages.